17.

Aim

To write a **Prolog program to find the sum of integers from 1 to N** using recursion.

Algorithm

1. Start the program.
2. Define the base case: when N = 0, the sum is 0.
3. For N > 0, calculate N - 1.
4. Recursively find the sum from 1 to N - 1.
5. Add N to the recursive sum.
6. Return the final sum.
7. Stop.

Prolog Program

```
% Sum of integers from 1 to N

sum(0, 0).

sum(N, S) :-
   N > 0,
   N1 is N - 1,
   sum(N1, S1),
   S is N + S1.
```

Sample Query

```
?- sum(5, S).
```

Sample Output

```
S = 15.
```

Result

The Prolog program was successfully implemented to calculate the **sum of integers from 1 to N** using recursion.

18.

Aim

To write a **Prolog program to create a database containing Name and Date of Birth (DOB)** and retrieve the details using queries.

Algorithm

1. Start the program.
2. Define facts using the predicate person(Name, DOB).
3. Store the name and date of birth of each person.
4. Use Prolog queries to retrieve the required person's details.
5. Display the result.
6. Stop.

Prolog Program

```
% Database with Name and Date of Birth

person(aryan, '14-04-2005').

person(ravi, '10-06-2004').

person(rahul, '25-12-2003').

person(priya, '15-08-2005').

person(sneha, '20-02-2004').
```

Sample Queries

To find the DOB of Aryan:

```
?- person(aryan, DOB).
```

Output

DOB = '14-04-2005'.

Sample Queries 2 :

To find the name of a person born on 10-06-2004:

```
?- person(Name, '10-06-2004').
```

Output

Name = ravi.

19.

Aim

To write a **Prolog program to create a database containing Student, Teacher, and Subject information** and retrieve the details using queries.

Algorithm

1. Start the program.
2. Define facts for students.
3. Define facts for teachers.
4. Define facts showing which teacher teaches which subject.
5. Define facts showing which student studies which subject.
6. Use Prolog queries to retrieve the required information.
7. Display the result.
8. Stop.

Prolog Program

```
% Student Database


student(aryan).

student(ravi).

student(priya).


% Teacher Database


teacher(ramesh).

teacher(suresh).

teacher(anitha).


% Subject Database
```

```
subject(dbms).
subject(ai).
subject(cybersecurity).

% Teacher teaches Subject

teaches(ramesh, dbms).
teaches(suresh, ai).
teaches(anitha, cybersecurity).

% Student studies Subject

studies(aryan, dbms).
studies(ravi, ai).
studies(priya, cybersecurity).
```

Sample Queries

Find the subject studied by Aryan:

```
?- studies(aryan, Subject).
```

Output

Subject = dbms.

Find the teacher who teaches DBMS:

```
?- teaches(Teacher, dbms).
```

Output

Teacher = ramesh.

Result

The Prolog database was successfully implemented using **Student, Teacher, and Subject** facts, and the required information was retrieved using Prolog queries.

20.

Aim

To write a Prolog program to create a database of planets and retrieve information about planets using Prolog queries.

Algorithm

1. Start the program.
2. Define facts containing the names of planets.
3. Define the distance of each planet from the Sun.
4. Define the type of each planet.
5. Use Prolog queries to retrieve planet information.
6. Display the required result.
7. Stop.

Prolog Program

```
% Planet Database


planet(mercury).

planet(venus).

planet(earth).

planet(mars).

planet(jupiter).

planet(saturn).

planet(uranus).

planet(neptune).


% Distance from Sun in million km


distance(mercury, 58).

distance(venus, 108).
```

```
distance(earth, 150).

distance(mars, 228).

distance(jupiter, 778).

distance(saturn, 1430).

distance(uranus, 2870).

distance(neptune, 4495).

% Type of Planet

type(mercury, terrestrial).

type(venus, terrestrial).

type(earth, terrestrial).

type(mars, terrestrial).

type(jupiter, gas_giant).

type(saturn, gas_giant).

type(uranus, ice_giant).

type(neptune, ice_giant).
```

Sample Queries

Find the type of Earth:

?- type(earth, Type).

Output

Type = terrestrial.

Find the distance of Mars from the Sun:

?- distance(mars, Distance).

Output

Distance = 228.

Find all terrestrial planets:

?- type(Planet, terrestrial).

Output

Planet = mercury ;

Planet = venus ;

Planet = earth ;

Planet = mars.

Result

The Planet Database was successfully implemented in Prolog, and planet information was retrieved using suitable queries.